

**Q5)** Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter  $p$ , substitute the ciphertext letter  $C$ :  $C = E([a, b], p) = (ap + b) \bmod 26$ . A basic requirement of any encryption algorithm is that it be one-to-one. That is, if  $p \neq q$ , then  $E(k, p) \neq E(k, q)$ . Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of  $a$ . For example, for  $a = 2$  and  $b = 3$ , then  $E([a, b], 0) = E([a, b], 13) = 3$ .

- a. Are there any limitations on the value of  $b$ ?
- b. Determine which values of  $a$  are not allowed.

**Aim:** To write a C program to implement the **Affine Caesar Cipher** using the encryption formula  $C = (aP + b) \bmod 26$ , and determine the valid values of  $a$  and  $b$  for successful encryption and decryption.

**Algorithm:**

1. Start the program.
2. Declare variables for plaintext, ciphertext, keys  $a$  and  $b$ , and loop counter.
3. Read the plaintext, value of  $a$ , and value of  $b$  from the user.
4. Check whether  $a$  is coprime with 26 (i.e.,  $\text{GCD}(a, 26) = 1$ ). If not, display an error message and terminate the program.
5. Traverse each character of the plaintext.
6. Convert each alphabetic character into its numerical equivalent ( $A = 0, B = 1, \dots, Z = 25$ ).
7. Encrypt each character using the formula:  $C = (a \times P + b) \bmod 26$ .
8. Convert the encrypted numerical value back to its corresponding alphabet and store it in the ciphertext.
9. Display the encrypted ciphertext.
10. Stop the program..

**Result:**

Thus, the C program to implement the **Affine Caesar Cipher** was successfully executed. It was observed that  $b$  can take any value from **0 to 25**, whereas  $a$  must be **coprime with 26** to ensure a unique and reversible encryption process.

## Answer to the Questions

**a. Are there any limitations on the value of  $b$ ?**

**Answer:**

No. The value of  $b$  can be **any integer from 0 to 25** because it simply shifts the encrypted letters after multiplication. It does not affect whether the cipher is reversible.

**b. Determine which values of  $a$  are not allowed.**

**Answer:**

The value of  $a$  must be **coprime with 26**, i.e.,  $\text{GCD}(a, 26) = 1$ .

The **allowed values** of  $a$  are:

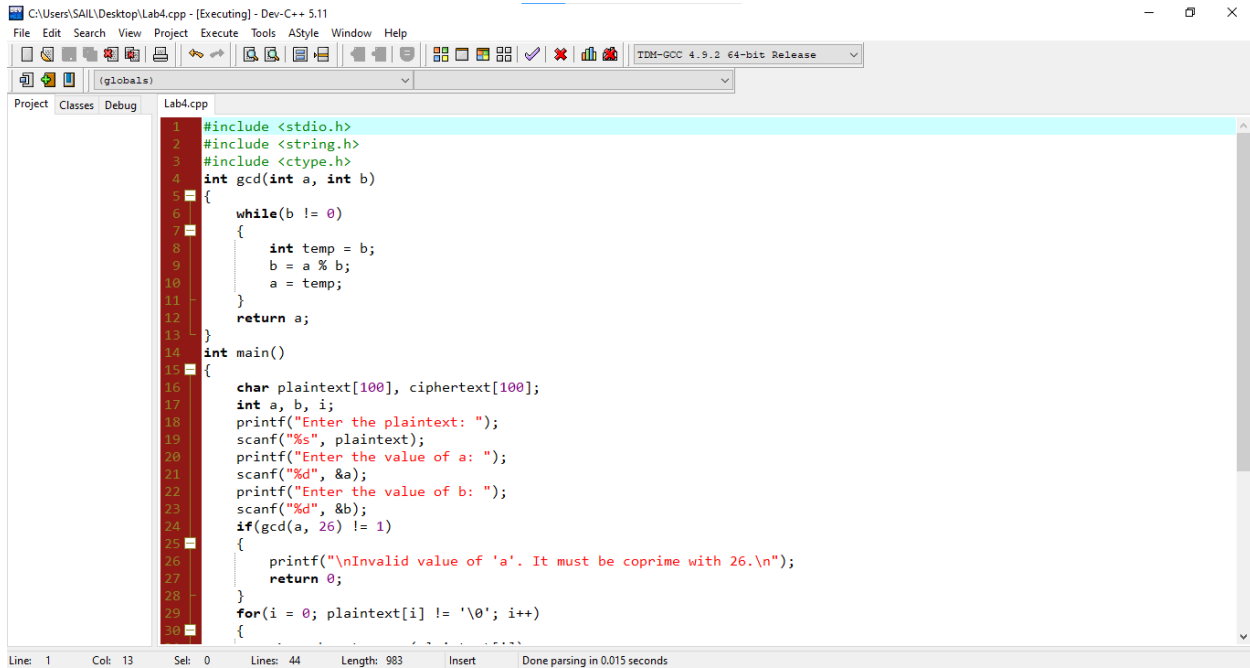
**1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25**

The **not allowed values** are:

**0, 2, 4, 6, 8, 10, 12, 13, 14, 16, 18, 20, 22, 24**

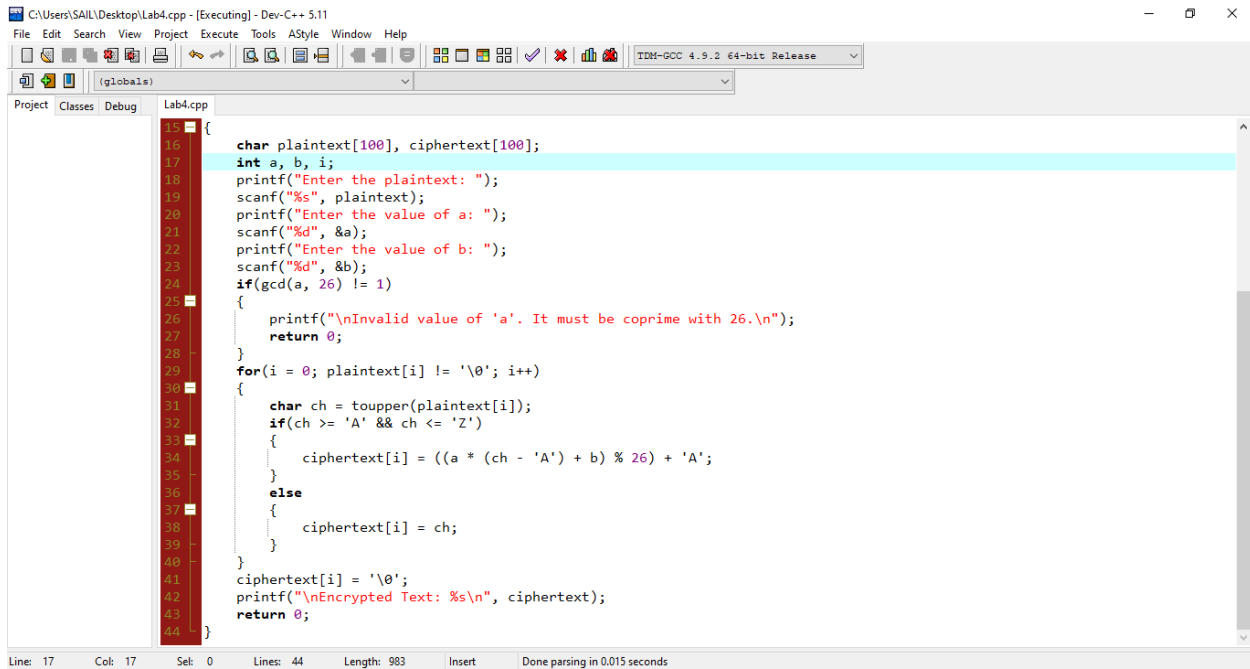
These values share a common factor with 26, making the encryption **not one-to-one**, so decryption becomes impossible.

## Code:



```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 int gcd(int a, int b)
5 {
6     while(b != 0)
7     {
8         int temp = b;
9         b = a % b;
10        a = temp;
11    }
12    return a;
13 }
14 int main()
15 {
16     char plaintext[100], ciphertext[100];
17     int a, b, i;
18     printf("Enter the plaintext: ");
19     scanf("%s", plaintext);
20     printf("Enter the value of a: ");
21     scanf("%d", &a);
22     printf("Enter the value of b: ");
23     scanf("%d", &b);
24     if(gcd(a, 26) != 1)
25     {
26         printf("\nInvalid value of 'a'. It must be coprime with 26.\n");
27         return 0;
28     }
29     for(i = 0; plaintext[i] != '\0'; i++)
30     {
```

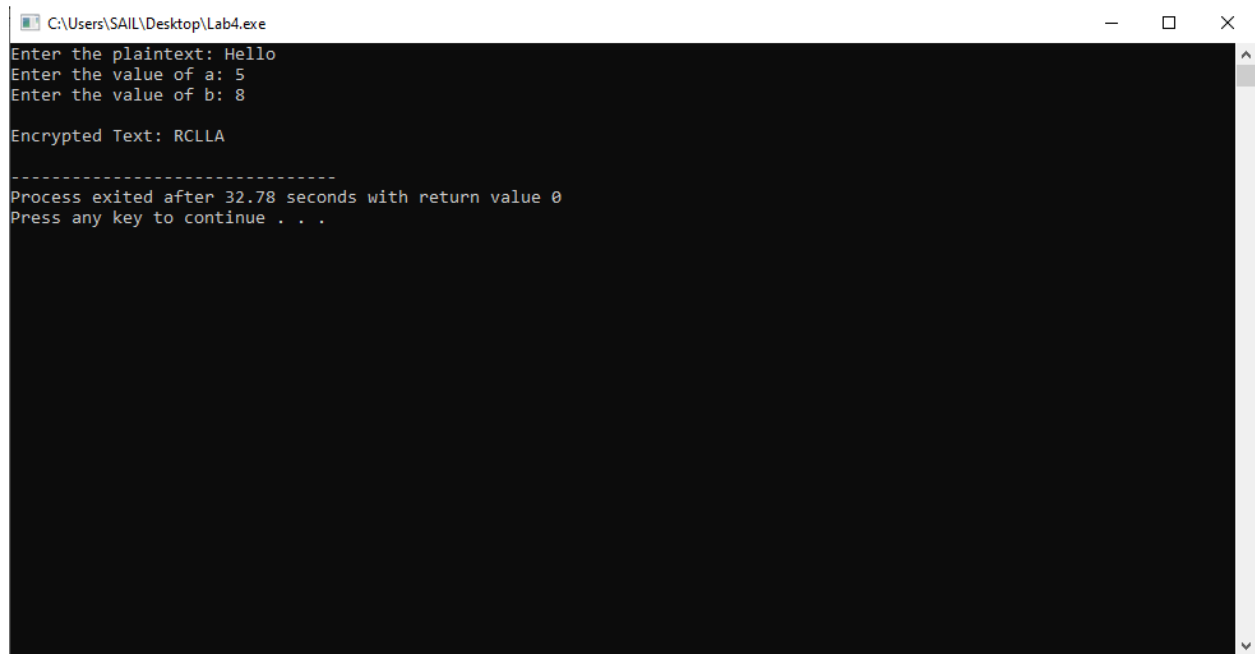
Line: 1 Col: 13 Sel: 0 Lines: 44 Length: 983 Insert Done parsing in 0.015 seconds



```
15 {
16     char plaintext[100], ciphertext[100];
17     int a, b, i;
18     printf("Enter the plaintext: ");
19     scanf("%s", plaintext);
20     printf("Enter the value of a: ");
21     scanf("%d", &a);
22     printf("Enter the value of b: ");
23     scanf("%d", &b);
24     if(gcd(a, 26) != 1)
25     {
26         printf("\nInvalid value of 'a'. It must be coprime with 26.\n");
27         return 0;
28     }
29     for(i = 0; plaintext[i] != '\0'; i++)
30     {
31         char ch = toupper(plaintext[i]);
32         if(ch >= 'A' && ch <= 'Z')
33         {
34             ciphertext[i] = ((a * (ch - 'A') + b) % 26) + 'A';
35         }
36         else
37         {
38             ciphertext[i] = ch;
39         }
40     }
41     ciphertext[i] = '\0';
42     printf("\nEncrypted Text: %s\n", ciphertext);
43     return 0;
44 }
```

Line: 17 Col: 17 Sel: 0 Lines: 44 Length: 983 Insert Done parsing in 0.015 seconds

## Output:



```
C:\Users\SAIL\Desktop\Lab4.exe
Enter the plaintext: Hello
Enter the value of a: 5
Enter the value of b: 8

Encrypted Text: RCLLA

-----
Process exited after 32.78 seconds with return value 0
Press any key to continue . . .
```